

Tổng Hợp Kiến Thức

1. Tại sao Validation Quan trọng?

- **Bảo mật:** Ngăn chặn tấn công như SQL Injection, XSS, DoS.
- **Toàn vẹn Dữ liệu:** Đảm bảo dữ liệu chính xác, nhất quán (VD: email hợp lệ, giá dương).
- **Ổn định Ứng dụng:** Tránh lỗi do dữ liệu không hợp lệ.
- **Trải nghiệm Người dùng:** Phản hồi lỗi rõ ràng, dễ sửa.
- **Client-side vs Server-side:**
 - **Client-side:** Nhanh, cải thiện UX nhưng không đáng tin (dễ bị bypass).
 - **Server-side:** Bắt buộc, an toàn, lớp phòng thủ cuối cùng.

2. Giới thiệu **express-validator**

- Middleware cho Express.js để **kiểm tra (validate)** và **làm sạch (sanitize)** dữ liệu request.
- Dựa trên thư viện **validator.js**.
- Tích hợp mượt mà với route của Express.
- Cung cấp nhiều validator/sanitizer có sẵn + hỗ trợ logic tùy chỉnh.

Cài đặt

```
npm install express-validator
```

- Kiểm tra: Đã thêm vào **package.json** trong **dependencies**.

3. Thiết lập Validation Cơ bản

Các Hàm Chính

- **check(field)** : Kiểm tra field ở mọi nơi (**body** , **query** , **params** , v.v.).
- **body(field)** : Kiểm tra **req.body** (phù hợp cho POST/PUT).
- **param(field)** : Kiểm tra **req.params** (VD: **/api/:id**).
- **query(field)** : Kiểm tra **req.query** (VD: **?page=1**).
- **cookie(field)** : Kiểm tra **req.cookies** .
- **header(field)** : Kiểm tra **req.headers** .

Validator Phổ biến

- **.notEmpty()** : Đảm bảo field không rỗng.
- **.isLength({ min, max })** : Kiểm tra độ dài chuỗi.
- **.isEmail()** : Kiểm tra định dạng email.
- **.isInt({ min, max })** : Đảm bảo là số nguyên trong khoảng.
- **.isFloat({ min, max })** : Đảm bảo là số thực trong khoảng.
- **.optional({ nullable: true })** : Cho phép field rỗng hoặc null.
- **.trim()** : Loại bỏ khoảng trắng đầu/cuối.

Xử lý Lỗi

- Dùng **validationResult(req)** để lấy lỗi.
- Định dạng lỗi: **[{ msg, param, location, value }]** .
- Trả lỗi: **res.status(400).json({ errors: errors.array() })** .

Ví dụ: POST **/api/categories**

```
const { body, validationResult } = require('express-validator');

router.post('/',
[
  body('name')
    .notEmpty().withMessage('Tên không được để trống')
    .isLength({ min: 3, max: 255 }).withMessage('Tên phải từ 3-255 ký tự')
    .trim(),
  body('description')
    .optional()
    .isLength({ max: 500 }).withMessage('Mô tả tối đa 500 ký tự')
    .trim()
],
(req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  // Tiếp tục logic (VD: Category.create)
}
);
```

4. Xử lý Lỗi Tập trung (Tùy chọn)

- Tạo middleware để xử lý lỗi validation chung.

```
// middlewares/validationErrorHandler.js
const { validationResult } = require('express-validator');

const handleValidationErrors = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  next();
};

module.exports = handleValidationErrors;
```

- Sử dụng trong route:

```
const handleValidationErrors = require('../middlewares/validationErrorHandler');
router.post('/', [/* quy tắc validation */], handleValidationErrors, async (req, res, next) => { /* logic */ });
```

5. Custom Validation

- Dùng `.custom()` để tạo logic tùy chỉnh (VD: kiểm tra DB).
- Trả về `true` hoặc ném lỗi/reject Promise.
- Ví dụ: Kiểm tra `categoryId` tồn tại trong DB.

```
body('categoryId')
  .optional({ nullable: true })
  .isInt({ min: 1 }).withMessage('ID danh mục phải là số nguyên dương')
  .custom(async (value) => {
    if (value) {
      const category = await Category.findByPk(value);
      if (!category) throw new Error('Danh mục không tồn tại');
    }
  })
```

```
    }  
    return true;  
  });
```

6. Validate Các Kiểu Dữ liệu Khác

- **Số:**
 - `.isInt({ min: 1 })` : Số nguyên dương (VD: ID).
 - `.isFloat({ min: 0 })` : Số thực không âm (VD: giá).
 - `.toFloat()` : Chuyển thành số thực.
- **Params** (VD: `/api/products/:id`):

```
param('id').isInt({ min: 1 }).withMessage('ID phải là số nguyên dương');
```

- **Query** (VD: `?page=1`):

```
query('page').isInt({ min: 1 }).withMessage('Trang phải là số nguyên dương');
```

Ví dụ: POST `/api/products`

```
router.post('/',  
  [  
    body('name').notEmpty().withMessage('Tên không được để trống').isLength({ min: 3, max: 255 }).trim(),  
    body('price').notEmpty().isFloat({ min: 0 }).withMessage('Giá phải không âm').toFloat(),  
    body('categoryId')  
      .optional({ nullable: true })  
      .isInt({ min: 1 })  
      .custom(async (value) => {  
        if (value) {  
          const category = await Category.findByPk(value);  
          if (!category) throw new Error('Danh mục không tồn tại');  
        }  
        return true;  
      })  
  ],  
  handleValidationErrors,  
  async (req, res, next) => {  
    // Logic tạo sản phẩm  
  }  
);
```

7. Refactor Quy tắc Validation

- Tách quy tắc ra file riêng để dễ tái sử dụng.
- Ví dụ: `validators/productValidator.js`

```
const { body, param } = require('express-validator');  
const { Category } = require('../models');  
  
const createProductValidationRules = () => [  
  body('name').notEmpty().withMessage('Tên không được để trống').isLength({ min: 3, max: 255 }).trim(),  
  body('price').notEmpty().isFloat({ min: 0 }).withMessage('Giá phải không âm').toFloat(),  
  body('categoryId')  
    .optional({ nullable: true })  
    .isInt({ min: 1 })  
    .custom(async (value) => {
```

```

    if (value) {
      const category = await Category.findByPk(value);
      if (!category) throw new Error('Danh mục không tồn tại');
    }
    return true;
  })
];

const commonIdParamValidation = () => [
  param('id').isInt({ min: 1 }).withMessage('ID phải là số nguyên dương')
];

module.exports = { createProductValidationRules, commonIdParamValidation };

```

- Sử dụng trong route:

```

const { createProductValidationRules } = require('../validators/productValidator');
router.post('/', createProductValidationRules(), handleValidationErrors, async (req, res, next) => { /* logic */
});

```

8. Best Practices

- Luôn validate phía server, dù đã có client-side.
- Dùng validator cụ thể (`body` , `param`) thay vì `check` chung.
- Chuỗi validator hợp lý; dùng `.withMessage()` để rõ ràng.
- Làm sạch input (VD: `.trim()` , `.toFloat()`).
- Xử lý lỗi nhất quán (nên dùng middleware chung).
- Tách quy tắc validation cho dự án lớn.
- Kết hợp với ràng buộc Sequelize để bảo vệ dữ liệu.